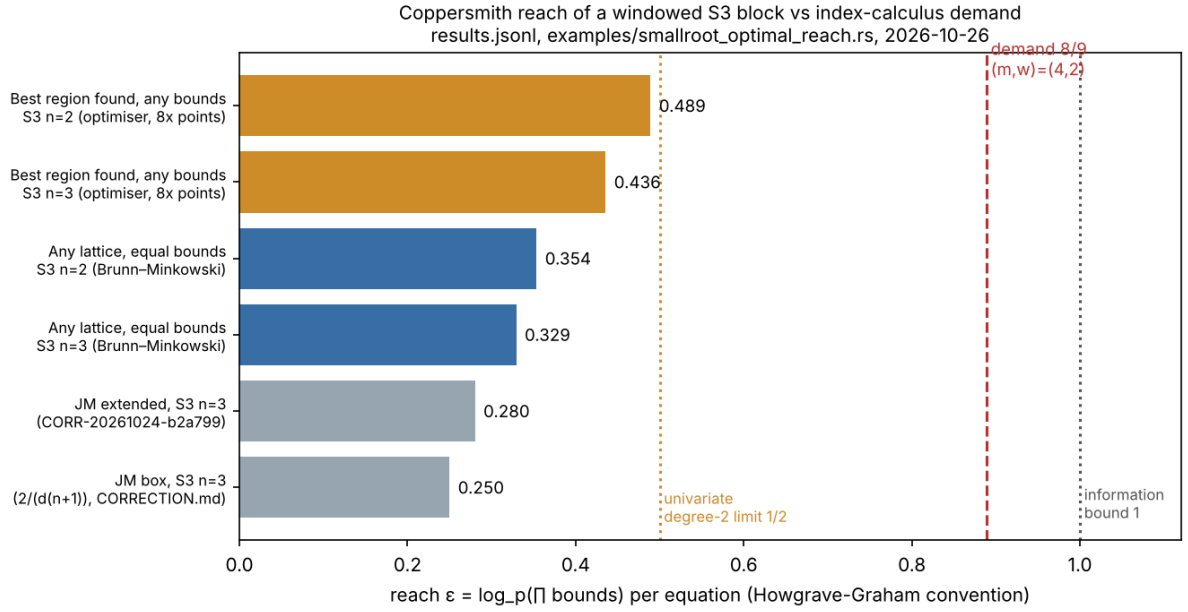
Small-root reach bound for windowed S3 blocks

Small-root decomposition: a strategy-independent reach bound for windowed S_3 blocks

2026-10-26 · branch `claude/smallroot-optimal-reach` · tool `examples/smallroot_optimal_reach.rs` (std only) · records `results.jsonl` · graph `reach_vs_demand.svg` · PDF `report.pdf`

Class: analysis (exact computation plus a numerical optimisation of a bound). Not an index-calculus measurement, not an improvement, no leaderboard row. Lead: `LEAD-ICP-coppersmith-smallroot-semaev` (crypto-autoresearcher `knowledge/leads/ecdlp/ic-prime/`). Item 6 of the 2026-10-26 triage (`docs/session-logs/2026-10-26.md`).



Verdict

The lead's decided next step was: compute the *provably optimal* shift set (Ryan, EUROCRYPT 2025, ePrint 2024/1577) for the S_3 block and report whether its reach clears the index-calculus demand `ε_required(4,2) = 8/9`. This study answers the question for **every** Coppersmith lattice built from `(p, f)^t`, the optimal one included, without building Ryan's lattice:

quantity	value	status
demand `ε_required(4,2)` (unchanged by CORR-20261024-b2a799)	**0.8889**	derived (CLOSURE.md §1)
JM extended `ε(S₃)`, HG convention (CORR-20261024-b2a799)	0.2803	recorded there
any lattice, equal bounds, S_3 in 3 unknowns: `(n! ·vol N)^{−1/n}`	**0.3293**	proved here (§2), `vol N = 14/3` exact
any lattice, equal bounds, S_3 in 2 unknowns	**0.3536**	proved here, `vol N = 4` exact
best region found, any per-variable bounds, S_3 in 3 unknowns	0.4357 (→ 1/2)	numerical (§3)
best region found, any per-variable bounds, S_3 in 2 unknowns	0.4892 (→ 1/2)	numerical (§3)
exact lattice gain equals the bound (attained, not slack)	11 / 11 cases	exact (§4)

So the windowed-decomposition family is closed **independently of the shift strategy**: with equal per-variable bounds no lattice reaches more than 0.33 per S_3 block against a demand of 0.89, and with unequal bounds the optimiser finds nothing above the univariate degree-2 value 1/2. The atlas entry still quotes the superseded margin −0.048; the corrected margin is about −0.6 (JM) and at best about −0.39 for any strategy within the search (1/2 against 8/9; −0.56 proved for equal bounds).

1. Setting and what “any lattice” means

A block is $f = S_3(x_1, x_2, x_3) \bmod p$ with $|x_i| < p^{\delta_i}$; the reach is $\varepsilon = \Sigma\ \delta_i$, the convention of `analysis/smallroot-reach/CORRECTION.md` (crypto-autoresearcher). A Coppersmith lattice on a monomial set M consists of polynomials of $(p, f)^t$ supported on M; Ryan’s optimal shifts, Jochemsz–May basic and extended, and every hand-made strategy are sublattices of

$$L(M) = \{\ g : \mathrm{supp}\ g \subseteq M,\ g \in (p,f)^t + p^t\, Z[x]\ \}.$$

Write $E(M) = \log_p [L(M) : p^t\, Z^M]$. Howgrave-Graham ($\det L < p^{t \cdot \dim L}$, dimension factors dropped, which favours the attack) is $\Sigma_{\mu \in M} \langle \delta, \mu \rangle < E(M)$.

2. The bound

1. **Layers.** p, f is a regular sequence in $Z_p[x]$, so $((p,f)^t : p^i) = (p,f)^{t-i}$; layer i of $L(M)$ reduces mod p into $\bar{f}^{t-i} \cdot F_p[x] \cap \mathrm{span}\ M$. Hence $E(M) \le \Sigma_{s=1}^{t} \dim(\bar{f}^s\ F_p[x] \cap \mathrm{span}\ M)$.
2. **Ostrowski.** $\mathrm{Newt}(h \cdot f^s) = \mathrm{Newt}(h) + s \cdot N$ with $N = \mathrm{Newt}(f)$. For M the lattice points of a convex R, $\mathrm{supp}(h\ f^s) \subseteq M$ forces $\mathrm{supp}\ h \subseteq R \ominus sN$, so $\dim(\ldots) \le \#(R \ominus sN) \cap Z^n$ (the *translate count*).
3. **Brunn–Minkowski + bathtub.** $(R \ominus sN) + sN \subseteq R$ gives $\mathrm{vol}(R \ominus sN) \le (\mathrm{vol}\ R^{1/n} - s \cdot \mathrm{vol}\ N^{1/n})^n$, so $E \le \mathrm{vol}\ R^{(n+1)/n} / ((n+1) \cdot \mathrm{vol}\ N^{1/n})$; among regions of given volume in the orthant, $\int \langle \delta, x \rangle$ is smallest on a simplex. Together, for equal bounds, $\boldsymbol{\varepsilon \le (n! \cdot \mathrm{vol}\ N)^{-1/n}}$. It is tight on both calibrations: 1 for a linear form in two unknowns, $1/d$ univariate.

Scope of the proof: convex monomial sets, equal per-variable bounds, the ideal $(p, f)^t$ of one block. For unequal bounds the inequality bounds only $\prod \delta_i$, so §3 is numerical.

3. Unequal bounds: region optimisation

`optimize` mode maximises the translate-count reach over down-closed polytopes $R = \{x \ge 0 : a_j \cdot x \le 1\}$ (3–10 facets, positive normals) **and** over the direction of the per-variable bounds, by random-restart hill climbing (6 restarts, 2,500 steps). Each region is dilated to a fixed lattice-point budget; every value is therefore the exact translate count of a real finite monomial set, a valid bound at that M, and approaches the continuum value from below as the budget grows (the 8× column). Calibrations first: a linear form in two unknowns must reach 1 and a univariate quadratic 1/2.

f	unknowns	facets	seed	reach (budget)	reach (8× points)	bound direction
linear	2	3	11	1.0001	1.0000	[1.0, 0.0]
univariate-d2	1	1	12	0.4999	0.5000	[1.0]
s3	3	3	21	0.4049	0.4331	[0.0, 1.0, 0.0]
s3	3	6	22	0.4055	0.4356	[0.0, 0.0, 1.0]
s3	3	10	23	0.4048	0.4349	[0.9998, 0.0001, 0.0001]
s3	3	6	24	0.4056	0.4357	[0.0, 0.0, 1.0]
s3	3	10	25	0.4052	0.4352	[0.0001, 0.0, 0.9999]
s3	2	4	31	0.4807	0.4892	[1.0, 0.0]
s3	2	8	32	0.4807	0.4890	[1.0, 0.0]

Measured observation: every S_3 run, five seeds and 3–10 facets in three unknowns, two in two unknowns, ends with the whole bound on **one** variable (the others pinned to $O(1)$), the univariate degree-2 configuration, and its value rises toward 1/2 with resolution (0.405 → 0.436; 0.481 → 0.489). No mixed allocation scored higher. Within this search the supremum is 1/2; it is not proved for unequal bounds (§2’s inequality bounds only $\prod \delta_i$).

4. Exact check: the bound is attained

`exact` mode builds $L(M)$ itself (all shifts $p^{t-k}\ x^a\ f^k$ inside a larger box, outside-M columns eliminated first, elimination over the local ring Z/p^t, $p = 1000003$) and compares its gain with the translate count.

f	unknowns	t	box side	superset side	E exact	translate count	equal-bound reach at this M
linear	2	2	4	7	25	25	0.5000
linear	2	3	6	9	77	77	0.5238
univariate-d2	1	3	6	12	9	9	0.4286
s3	2	1	4	8	9	9	0.1800
s3	2	2	4	8	10	10	0.2000
s3	2	2	6	10	34	34	0.2313
s3	2	3	6	10	35	35	0.2381
s3	2	3	8	12	83	83	0.2562
s3	3	1	3	5	8	8	0.0833
s3	3	2	4	6	28	28	0.1120
s3	3	2	5	7	72	72	0.1333

In all 11 cases the exact gain **equals** the translate count: the bound is attained by real lattices, so it is the optimum for these M, not a loose estimate.

5. What this does not cover

- **Joint lattices for the three-block `(4,2)` system.** The demand counts reach per equation over the whole tree (`mβ + Σγ ≤ ε(w+1)`); a single lattice for three equations in six unknowns is not bounded here.
- **Non-convex monomial sets**, and unequal bounds beyond the optimiser's search (down-closed polytopes with up to 10 facets, 6 restarts each).
- **Non-Coppersmith solvers** (anything that is not a lattice of polynomials in `(p, f)^t`).
- **The checkpoint gate.** `scripts/checkpoint.sh push` could not upload (no S3 token in this container); `.resume/RESUME.md` was filled in locally.

6. Run record

- Command: `smallroot_optimal_reach all > results.jsonl`, binary built by `rustc -O --edition 2021 -D warnings` (rustc from cargo 1.97.0) from `examples/smallroot_optimal_reach.rs` at this branch's head. Started 2026-10-26T16:17:58Z (`started.txt`); 4 m 11 s wall, single thread (`time.txt`).
- Host: shared 4-core, 15 GB cloud container; no isolation (none is needed: no timing is reported, every number is a count or a ratio of counts).
- Seeds: SplitMix 20261026 (exact), 11–32 (optimiser), fixed in the source.
- All modes completed; no failures. `cargo build --release --example` was started first and stopped after 40 minutes compiling the library; the example uses only `std`, so `rustc` builds the same source.

7. Reproduce

```
        rustc -O --edition 2021 -D warnings examples/smallroot_optimal_reach.rs -o
/tmp/smallroot_optimal_reach
        /tmp/smallroot_optimal_reach all > results.jsonl     # or: cargo run --release --example
smallroot_optimal_reach -- all
        python3 plot.py .                                    # lays out recorded values only
```